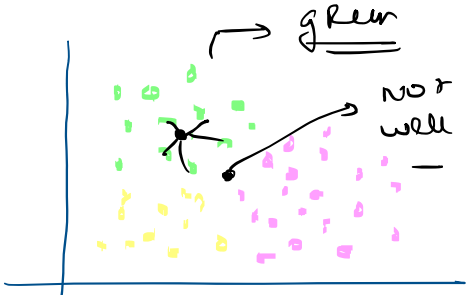
KNN – 2

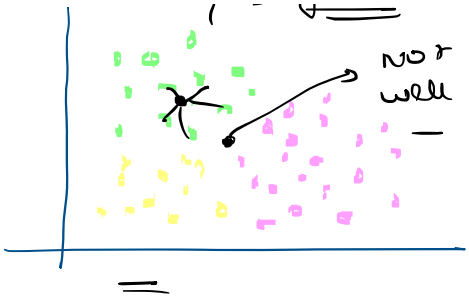
Agenda →

- → Recap
- → Bias & Variance
- → Time & Space complexity
- → KNN → Categorical features
- → Real world application
- → KNN imputation

- Recap → → How many ^nearest Neighbors
  - (*) KNN algorithm →
  - (*) Algo works →
    1. compute dist. (x_{query}, x^i_{train})
    2. sort the distances in asc
    3. take top $k = 5$ points → label
    4. Majority vote → class



[4 4] → Green

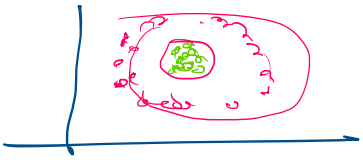


not a well behaved dp — 1 P.

4G 3 → Green

Tie ⇒ 4G 4P 4Y → Random class





well Behaved dp → similar kind of data

→ Homogeneous data ⇒

* KNN → Non parametric algo

→ everything happens at test time

→ Lazy learner ⇒

3 x 3

4x4

* Confusion matrix ⇒ → Pr., Recall, AUC

KNN

log Reg, DT, RF, Baggin

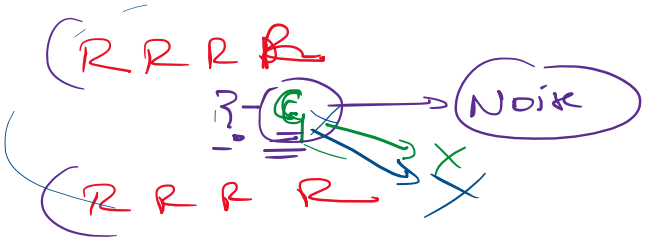
Bias & Variance Trade off ⇒

k = small, k = 1

→ OVERFITTING scenario

- low Bias
- High variance → k = 1

k = 1



Predⁿ = green

k = 1

k = 1

k = 3

(low Bias

100% accuracy

Train

100%

Test

High variance

Remove Training point

DATA → predⁿ = green

DATA → predⁿ = Red

1 DP → Removed

→ High variance

Val data → Best k

large value

Case 2 → K=100

underfitting

60 R
40 G → 100 DP

K=100 neighbours

K=100

G G G G G → G

?

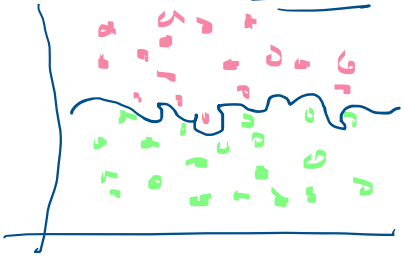
pred^n = Red

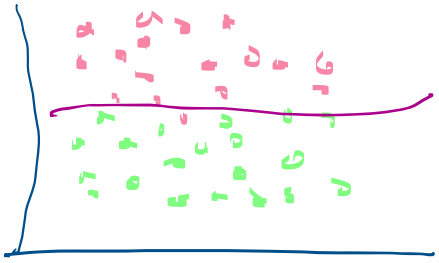
R R R → 60

K=100

K=1 overfit

snake model



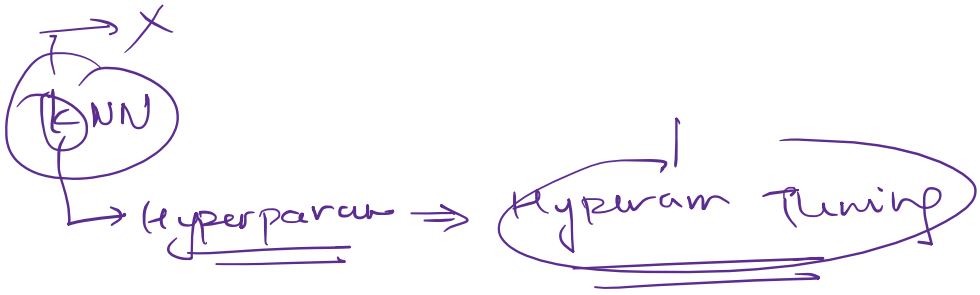


underfit → High Bias
→ low var.

K=N → underfit → majority class biased

Train → nearest neighborhood

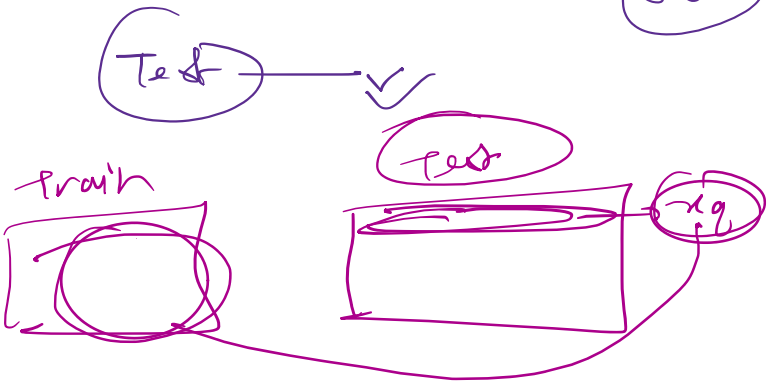
- Train → nearest neighborhood
- Val → Best k → Hyperparam Tuning
- Test → final eval.



model.fit(Train) ×



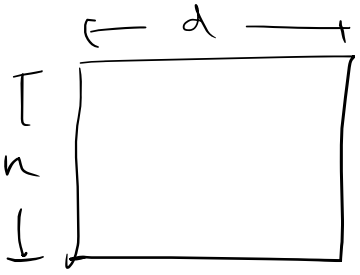
Val → Best k



- K = 5 nearest neighbors
- majority vote

Time complexity

- KNN → training ⇒ nothing
  ⇒ O(1) → constant



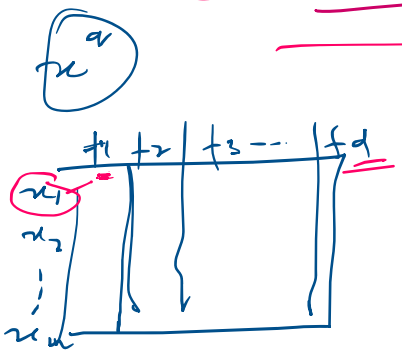
Training Phase
O(1) → constant
Stores the data

Testing Time complexity

$x_q \longrightarrow \vdots \longrightarrow \hat{y}_q$

1. find dist (x_q, all x^i)

- Distance
- $\sqrt{(x_1^q - x_1^i)^2 + (x_2^q - x_2^i)^2 + \cdots}$ d times



→

1 Distance → O(d)

n DP → O(nd)

.sort()

2. SORT distances ⇒ O(n log n)

np?

3. Top k elements ⇒ O(k) ✗ → constant

4. majority vote ⇒ O(1) ✗

Time complexity = O(nd + n log n)

✗✗

* Very High

Amazon search ⇒ if ! ⇒ ms

Testing

KNN → worst

* Biggest problem ⇒ V.V. slow *

n → h (Training)

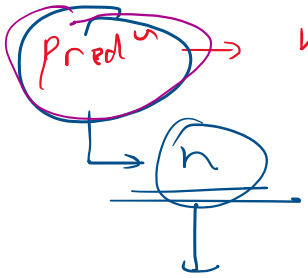
model

d

n

* Space complexity ⇒

Prod → how much storage do u



how much storage do u need

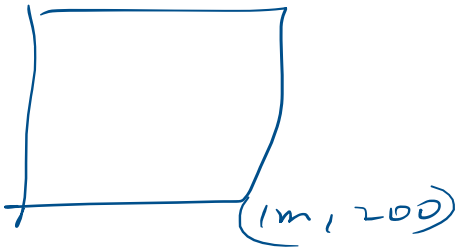
$\Rightarrow$ Huge STORAGE Req'

$O(n \times d)$

n Distances

$O(nd)$

model.fit(Train)



$\rightarrow$ Not Scalable

X



Logistic Reg^n

$x_q \longrightarrow \hat{y}_q$

model $\rightarrow$ learnt weights

$$\hat{y}_q = \sigma(w^T x_q + w_0)$$

$O(d+1)$

$\frac{1}{1+e^{-z}}$ O(1)

Predn → O(d)

Better than KNN

KNN — 10000 features — 10000 ⇒ — ms

model = [ ] w_0

O(d+1)

≈ O(d)

② Linear Reg^n ⇒

KNN → X Testing time

Log Reg^n → Testing

Log Reg^n with OVR ⇒ KNN

Quiz time!

Time Left: 30s

what to say if model has high varaince and low bias ?

23 users have participated

A	Model overfits	78%
B	Model underfits	22%

End Quiz Now

Quiz time!

Time Left: 13s

k→ hyperparameter, then what data to use for hyperparamter tuning ?

45 users have participated

A	validation	71%
B	training	20%
C	test	4%
D	entire data	4%

End Quiz Now

Quiz time!

Time Left: 15s

quiz(what do you think) What will be the training time complexity of kNN ?

31 users have participated

A	O(nd)	32%
B	O(n)	10%
C	O(nlogn)	16%
D	O(1)	42%

End Quiz Now

Quiz time!

Time Left: 31s

k-NN algorithm does more computation on test time rather than train time.

23 users have participated

A	TRUE	91%
B	FALSE	9%

End Quiz Now

KNN for categorical features

Data → Blood group →

A+
AB⁻
AB⁺
A⁺
O⁻
O

dist. B/w A+ & O⁻

Encoding ⇒ 0 & 1

label Encoding

2D → M → 0, F → 1

dist = 0

M → 0 → 1 →

F → 1

0 ↑ M, 1 ↑ F

M → F

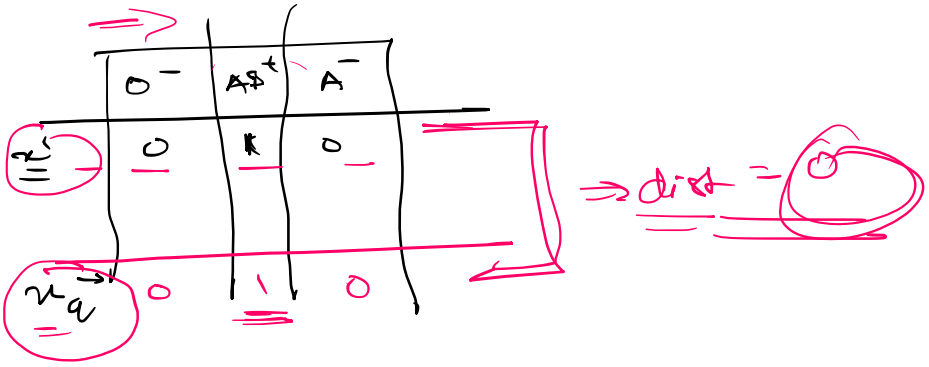
F − M = 1

F − F = 0

7 unique → OHE → 7 features

x_q

(0, 1, 0) – (0, 1, 0)



⇒ dist = 0

Pincode ⇒ Target encoding

1000 features

d ↑

* Euclidean dist? **

Why ⇒

2 dim^n

House (0, 0)

↓

2 neighbors → A(1, 1)

B → (10, 10)

$d(A) = \sqrt{1^2+1^2} = \sqrt{2}$

$d(B) = \sqrt{10^2+10^2} = \sqrt{200}$

d(A) = ...

↑ dim^n → scaling

100 features

A = [0.5, 0.5, 0.5 ---]

B = [0.6, 0.4, 0.5 ---]

C = [0.7, 0.3, 0.8 ---]

$$d = \sqrt{\sum_{i=1}^{100} (x_i - y_i)^2}$$

Nearest dist → 3.8

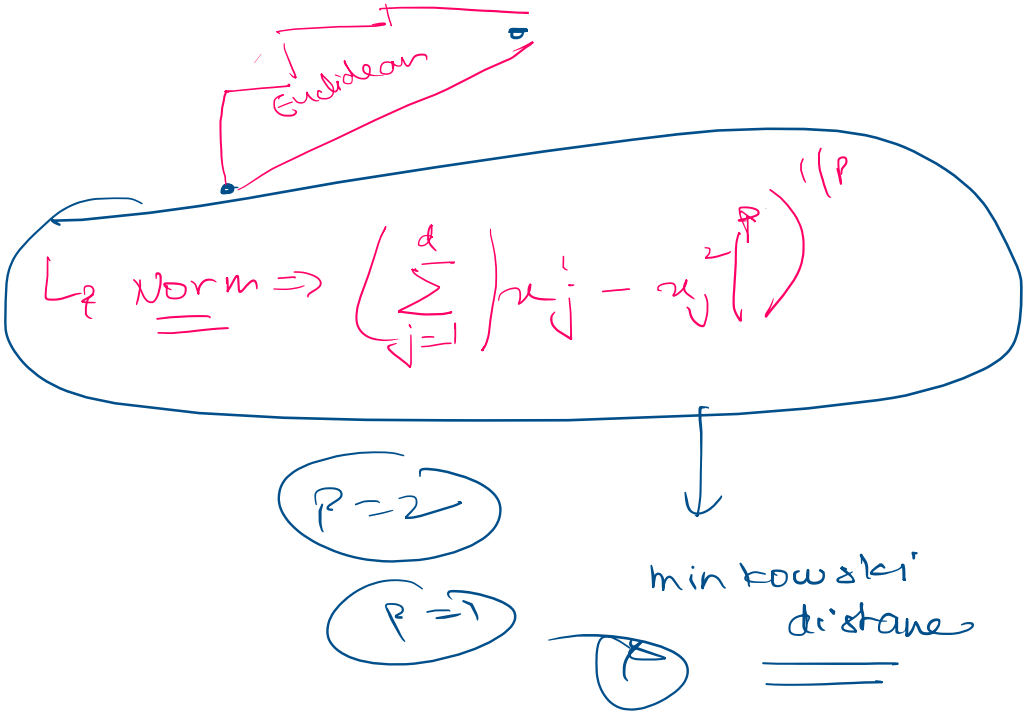
Farthest dist → 4.5

] ping dist.

(*) Euclidean distance fails → for higher dim^n ✗

* L_1 Norm ⇒ $\sum_{i=1}^{d} |x_i - y_i|$

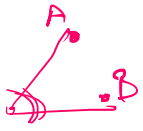
L_2 Norm $\Rightarrow$ $\sqrt{\sum_{j=1}^{d} (x_j^1 - x_j^2)^2}$



(+) Euclidean → fails

Cosine similarity

(+) $\cos sim(x_1, x_2) = \frac{x_1^T x_2}{\|x_1\| \|x_2\|}$



Range $\Rightarrow [-1, 1]$

(+) (+)

euclidean → asc.

distance ↑ ⇒ dissimilar items

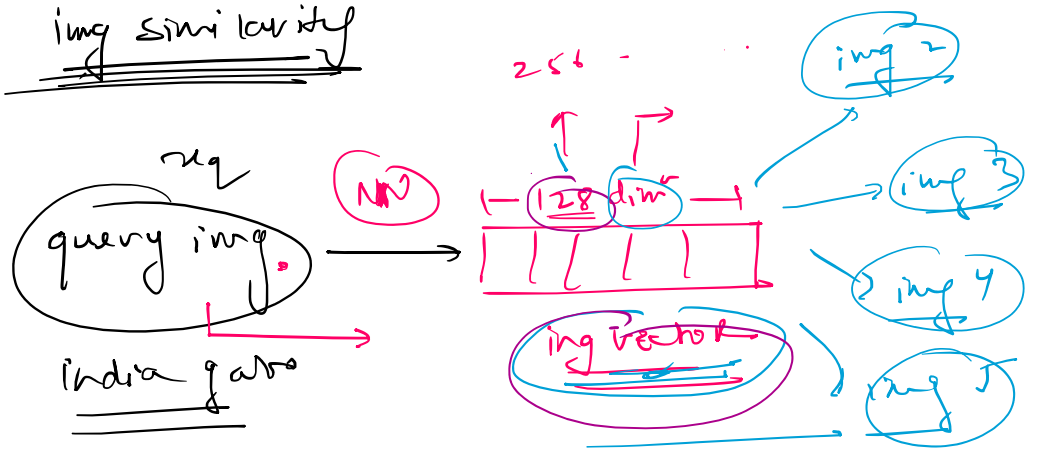
similarity ⇒ similarity

(7) desc → order

(4) higher dim → cosine

lower dim → minkowski / euclidean

(4) ⇒ Disadvantages

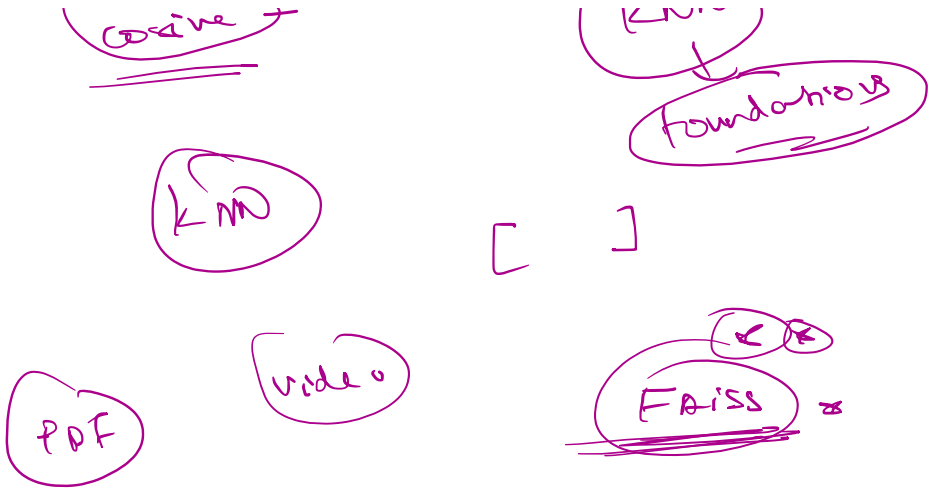


L2

top 5

cosine

KNN



Quiz time!

Time Left: 13s

quiz (what do you think) if One Hot Encoding increases data dimension to (d=1000), will Eculidean Distance work ?

42 users have participated

A	Yes	19%
B ✓	No	81%

End Quiz Now

(1,1,3) (1,3,5)

|1−1| + |3−1| + |5−3|

0 + 2 + 2

Quiz time!

Time Left: 38s

Which of the following will be Manhattan Distance between the two data point A(1,1,3) and B(1,3,5)?

16 users have participated

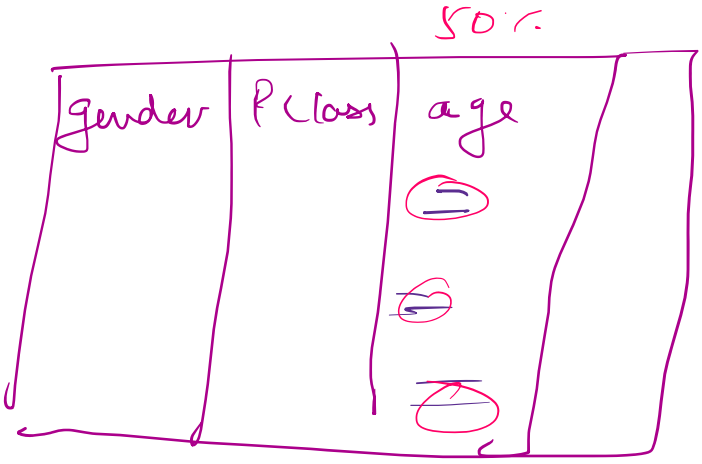
A	1	0%
B	2	13%
C ✓	4	81%
D	5	6%

End Quiz Now

Best use of KNN ⇒ imputation

KNN imputation

Titanic DATASet



missing values

↳ mean / median

29

* Smart way

↳ avg male & female age

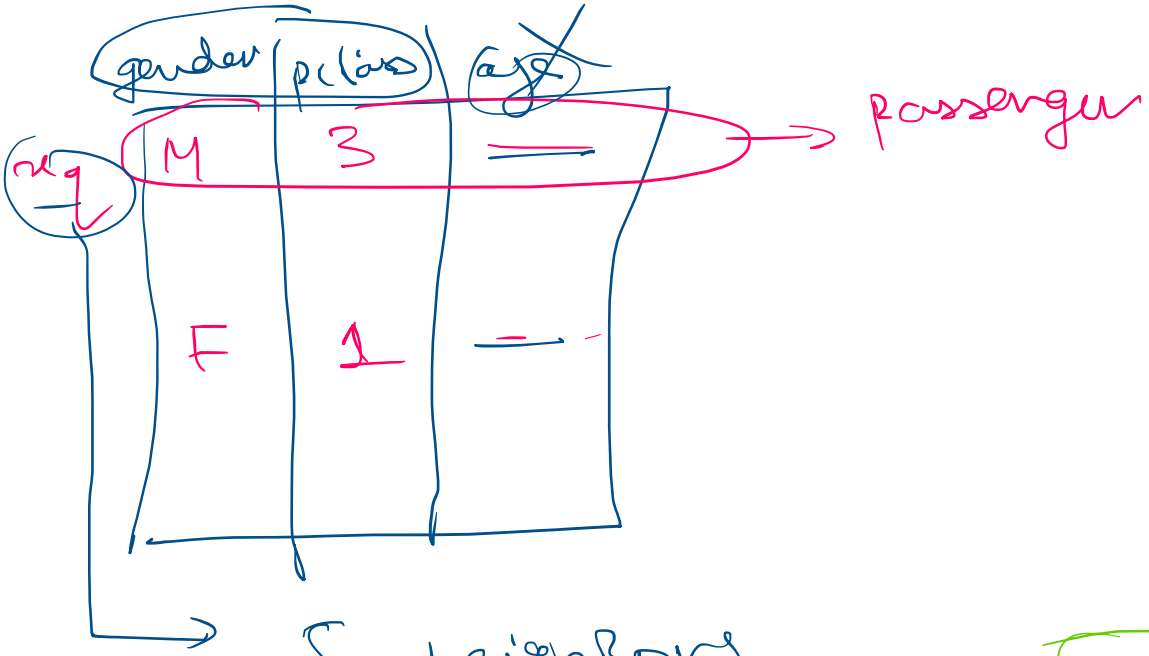
32 21

Pclass → I → 43

Pclass → II → 35

Pclass → I → 55

→ III → 26

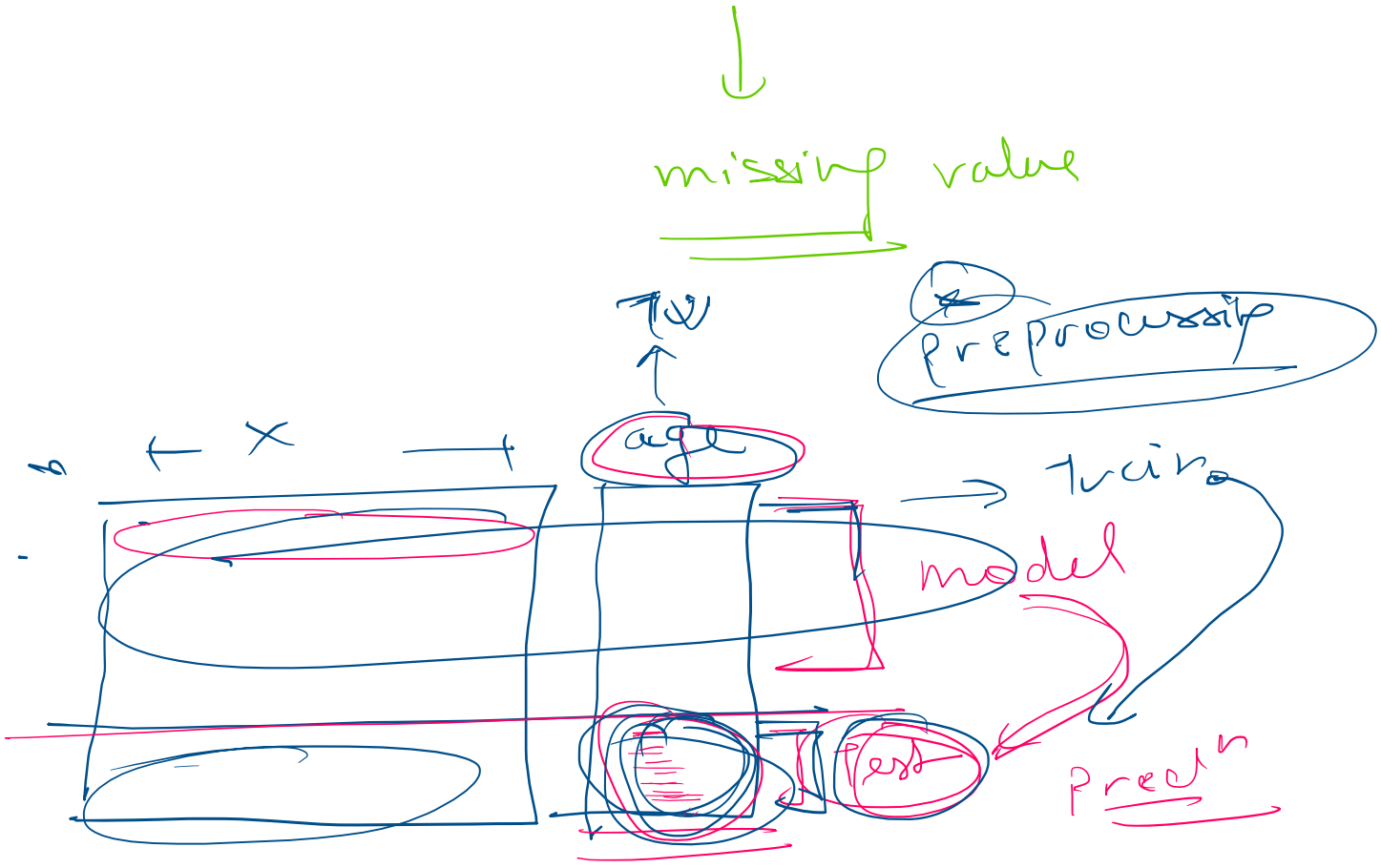


similar features

$x_q \rightarrow x_2, x_5, x_{18}, x_{38}, x_q$

age

mean



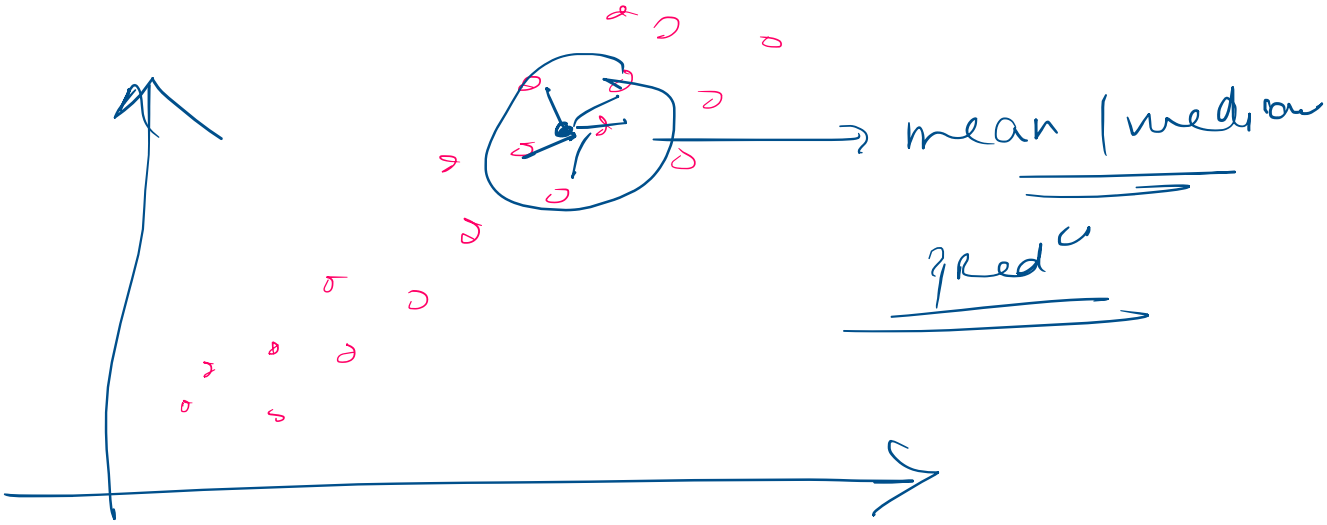
Model Based imputation

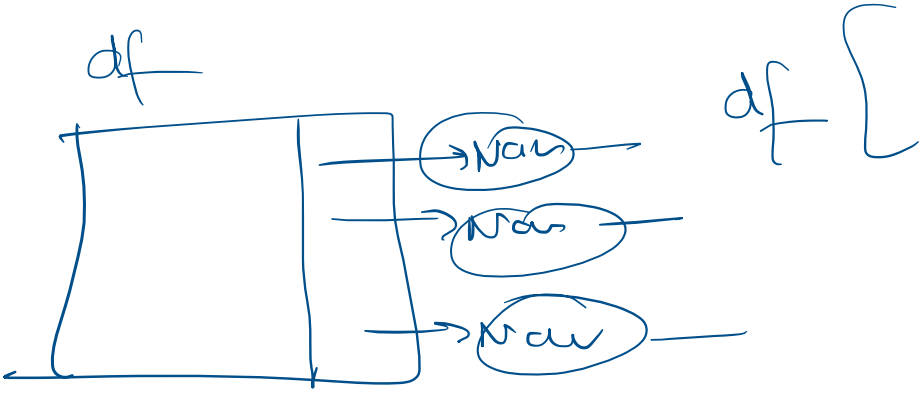
(time proces

missing → Preprocess → Hill

Regression → KNN

House price Pred^n





Train, val., Test ⇒

K-fold CV ⇒ Divide data into train &

K fold CV ⇒ into train & test

K=4

Train Test

800

train

test

85%

1st fold

Tr Tr Tr Val → train score, val score

2nd

Tr Tr Val Tr → 80% → train, val score

3rd

Tr Val Tr Tr → 60% → train, val score

4th

Val Tr Tr Tr → 82% → train, val score

↳ 1 single value of λ

λ=1

λ=1 ⇒ mode → 1

λ=2 → mode

train_score

train(1+2+3+4)/4

more reliable

more reliable

Small data → 60 20 20

Problem ⇒ computationally very Heavy

